Real-Time Lane Detection and Autonomous Steering

A comparison of classical pipelines and a polynomial-fit improvement

David Tovmasyan

Course project, May 2026

Where we are going

Motivation

The simulator

Three lane detectors

Two controllers

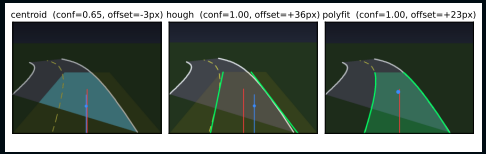
Experiments

Live demo

Discussion and conclusion

Why lane keeping (still) matters

- Most-deployed AD function: every modern car ships lane-keep assist or adaptive cruise.
- Two coupled problems
  - **Perception:** where is the lane?
  - **Control:** how do we stay in it?
- Learned segmenters dominate benchmarks, but a classical pipeline is:
  - interpretable (debuggable),
  - CPU-friendly (> 500 FPS),
  - zero training data required.



Same frame, three detectors.

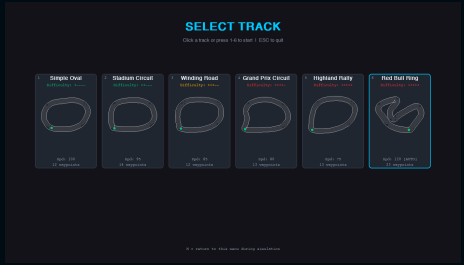
The concrete question

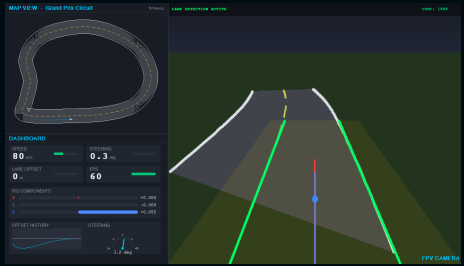
How much performance is left on the table
by the textbook Canny+Hough detector,
and can a **polynomial-fit upgrade**
close the gap on curved tracks
while staying **real-time**?

- Implement two classical baselines + one upgrade.
- Plug each into a PID controller.
- Add a Stanley controller driven from *ground truth* as a perception-free oracle (upper bound on what the controller can do given perfect perception).
- Evaluate on five tracks + a noise sweep.

What we built

- 2D forward-perspective driving simulator (Python + OpenCV + pygame).
- 5 tracks (difficulty 1–5), closed-loop Catmull–Rom spline.
- Camera: pinhole, 70° FoV, 18° pitch, 640×480 FPV.
- Vehicle: bicycle kinematics, $\dot{\theta} = (v/L)\tan\delta$.
- Three live panels: map, FPV overlay, dashboard.





Ground truth = analytical

Because we own the simulator, we can compute the things a benchmark is normally missing:

- **True lateral offset** from the centreline spline.
- **True heading error** vs. the track tangent.
- **True curvature** ahead (adaptive speed control, turn-sharpness grouping).
- **Ground-truth lane polygon** in the camera frame (re-project road from spline) $\Rightarrow$ **lane-IoU** against any predicted mask.

$\Rightarrow$ Every metric is anchored to a number the detector *cannot see*.

Baseline 1: centroid

Pipeline

1. Threshold road colour in HSV ($S < 40$, $40 < V < 110$).
2. Trapezoidal ROI mask.
3. Centroid of the resulting blob via image moments.
4. EMA smoothing across frames.

Why include it?

- A genuine *lower bound*: the simplest detector we could write down.
- Demonstrates how far the lane-line geometry adds value.

Spoiler

Does not complete a lap on any track.

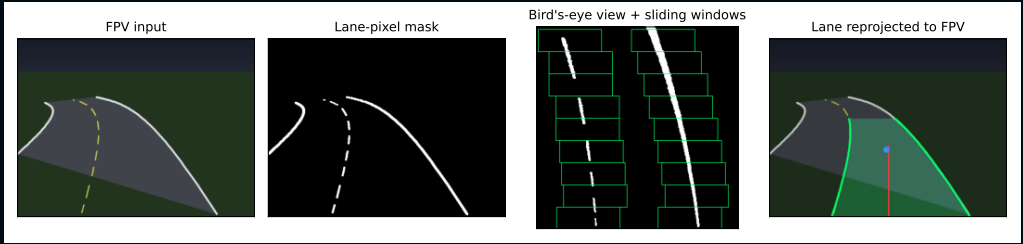
The blob centroid is biased away from the lane centre whenever the road is asymmetric in the field of view.

Baseline 2: Hough lines (the textbook pipeline)

1. Grayscale + 5×5 Gaussian blur.
2. Canny edges, $T_{\text{low}} = 50$, $T_{\text{high}} = 150$.
3. Trapezoidal ROI mask.
4. Probabilistic Hough ($\rho = 1$, $\theta = \pi/180$, thresh 25, minlen 30, gap 40).
5. Classify segments into left/right by slope sign + image side, length-weighted average per side.
6. Midpoint of the two lines at the bottom = lane centre. $\alpha = 0.4$ EMA smoothing.

- Works well on straights.
- **Wrong model on every curve:** a straight line fit through a bend underestimates the curvature.
- Fall-back: remember last good line on dropouts.

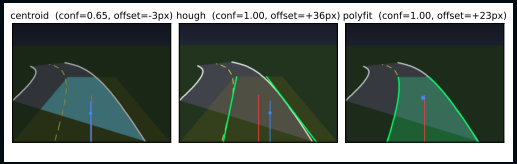
Proposed: bird's-eye + sliding-window polyfit



1. **Lane-pixel mask**: Sobel-x + HSV masks for white boundaries and yellow dashes.
2. **Inverse perspective**: homography to 320×320 BEV; curves become low-order polynomials.
3. **Sliding windows**: column histogram seeds two bases; nine 50-px windows climb upward.
4. **Quadratic fit + reprojection**: $x = ay^2 + by + c$ per lane, EMA on coefficients, reproject polygon to FPV.

The one idea that matters: look-ahead offset

- The polynomial fit gives us the lane shape, not just one point on it.
- Evaluate the lane centre at **45 % of the way up the BEV** – a 1-frame predictor essentially free on top of the polyfit.
- Feed that to the PID instead of the bottom-of-image offset.
- With it disabled, polyfit's RMS gain shrinks from $\sim 40\,\%$ to $\sim 10\,\%$.



The blue dot in the right panel is the reprojected look-ahead point.

PID – the baseline controller

$$\delta_t = K_p\, e_t + K_i \sum_{\tau \le t} e_\tau\, \Delta t + K_d\, \dot{e}_t$$

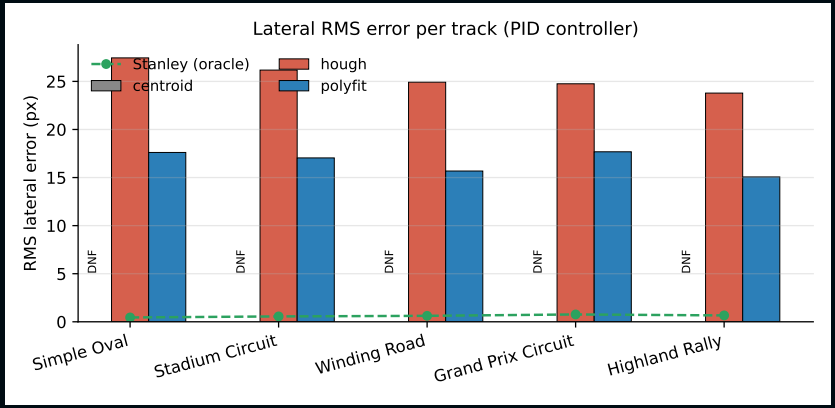
- Input: normalised lateral offset $e \in [-1, +1]$ from the detector.
- Gains tuned by hand on the oval: $K_p = 1.2$, $K_i = 0.008$, $K_d = 0.5$.
- Integral anti-windup at ± 50.
- Reactive only – has no model of the path.

Stanley – the perception-free oracle

$$\delta = \psi + \arctan\left(\frac{k\, e_{\perp}}{k_s + v}\right)$$

- Heading error ψ + cross-track error $e_{\perp}$ at the front axle, both computed from the *true* spline.
- Gains $k = 0.6$, $k_s = 20$.
- **It does not look at the camera.**
- Result on every track: < 1 px RMS lateral error.
- Interpretation: the gap between PID-on-detection and Stanley-on-truth is *purely* the perception error.

Headline result: RMS lateral error per track



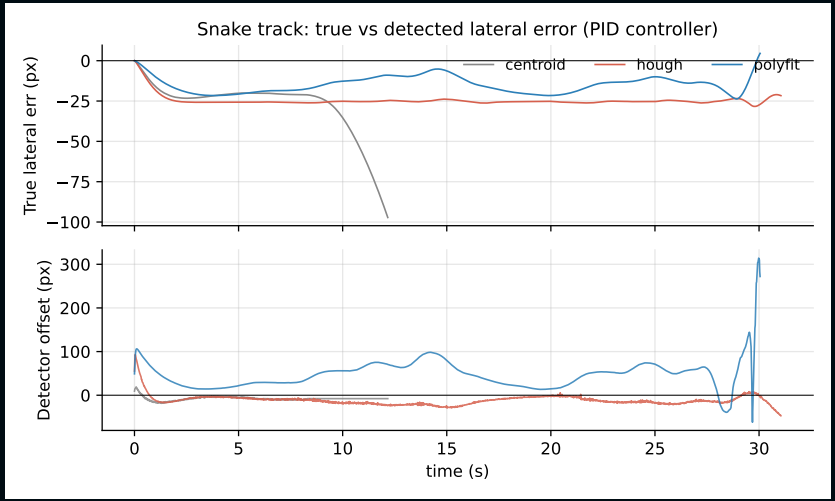
Centroid **DNF** on every track. Hough: 25.4 px mean RMS. Polyfit: **16.6** px (**-34 %**). Stanley+truth: < 1 px.

The full table

	HOUGH		POLYFIT		CENTROID	Stanley
Track	RMS	IoU	RMS	IoU	RMS	RMS
Simple Oval	27.4	0.17	17.6	0.25	DNF	0.4
Stadium Circuit	26.2	0.24	17.0	0.31	DNF	0.6
Winding Road	24.9	0.23	15.7	0.35	DNF	0.6
Grand Prix Circuit	24.8	0.26	17.7	0.36	DNF	0.8
Highland Rally	23.8	0.31	15.1	0.39	DNF	0.7
mean (px)	25.4	0.24	16.6	0.33	–	0.6

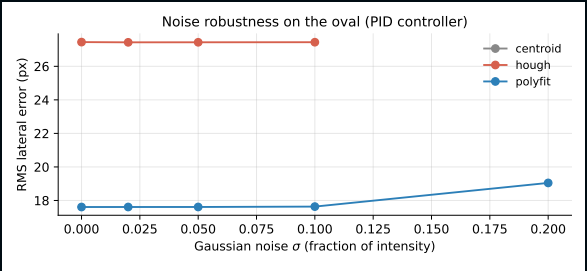
- Polyfit beats Hough on *every* track on both RMS and IoU.
- Lane-IoU rises from 0.24 to 0.33 on average (+**38** %) – the gain is geometric, not just from the look-ahead.

What the controller actually sees



Top: true error – polyfit bounded, Hough swings ±30 px. **Bottom:** detector offset (the PID input). Centroid (grey) leaves the road early.

Robustness to camera noise



- Same oval, PID, +Gaussian noise $\sigma \in [0, 0.20]$ per frame.
- Centroid never completes.
- Hough survives $\sigma \leq 0.10$, fails at 0.20 (Canny drowned out).
- **Polyfit completes at every σ** – its mask uses a *normalised* Sobel-x + colour.

Compute cost

Detector	ms / frame	equiv. FPS
centroid	1.7	$\sim$600
hough	0.7	$\sim$1450
polyfit	1.9	$\sim$520

- All three are comfortably real-time on a single CPU thread.
- Polyfit overhead is dominated by the perspective warp and `np.polyfit`, not by the sliding-window scan.
- Plenty of budget left for downstream tasks (other vehicles, obstacles, signs).

Live demo

```
python main.py --detector polyfit --controller pid
```

- Press **D** to cycle detectors live.
- Press **C** to toggle PID $\leftrightarrow$ Stanley.
- Press **M** to jump back to the track selector.
- Watch the dashboard: lane offset, FPS, PID components.

Suggested demo route: *Stadium* (Hough wobbles) $\rightarrow$ *Winding Road* (Hough struggles) $\rightarrow$ switch to polyfit $\rightarrow$ switch to Stanley.

What worked, what failed

Worked

- Look-ahead offset = single biggest gain.
- HSV + Sobel mask cleaner than Canny on dashed lines.
- Quadratic fits + EMA give stable curves.

Failed / limitations

- Centroid is structurally biased – no tuning saves it.
- Hough lingers on its last good line through tight bends.
- Polyfit's IPM warp assumes flat ground; real cameras need re-tuning.
- Simulator is too clean – numbers are an upper bound.

Take-aways

- A modest classical-CV upgrade (BEV warp + sliding-window quadratic fit) cuts RMS lateral error by **34–40 %** vs. the textbook Hough pipeline.
- The change is fully interpretable: we can point at each stage in Fig. 3 and say why it helps.
- Look-ahead offsets are essentially free on top of a polynomial fit and turn a reactive PID into a quasi-MPC.
- Closing the remaining gap to Stanley-on-truth would need sub-pixel-accurate perception $\Rightarrow$ a learned segmenter is the next step.

Thank you.

Code, simulator, report and these slides are all in this repo.
Questions?

Backup: detector confidence per track

- Centroid mean confidence: **0.27** (HSV blob coverage, near zero whenever the road leaves the trapezoid).
- Hough mean confidence: **0.99** (both lines almost always found – but "found" $\neq$ "correct").
- Polyfit mean confidence: **1.00** (both polynomial fits valid in every frame after warm-up).
- Take-away: high detector confidence does not imply low error. Lane-IoU is the better proxy.

Backup: why the look-ahead works

Bicycle kinematics:

$$\dot{\theta} = \frac{v}{L} \tan \delta$$

- A pure-P controller on e_{bottom} reacts to *current* position only.
- Adding the look-ahead error e_{LA} effectively augments the controller with a free derivative term that knows where the road is going.
- Equivalent to a one-step preview MPC with horizon $\sim 0.5\,\text{s}$ at typical speeds – without solving an optimisation.

Backup: reproducibility

```
git clone <repo>
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt matplotlib pdf2image
python experiments.py   # CSVs + summary figures
python make_qualitative_figures.py   # overlays
cd presentation && tectonic report.tex
cd presentation && tectonic slides.tex
```

All experiments are deterministic except the noise sweep, seeded by NumPy's default RNG.